

Homework 4

Caroline Hutchings

09/14/2026

Conceptual Questions

```
library(tidyverse) arrange(filter(select(as_tibble(iris), starts_with("Petal"), Species),
Petal.Length < 1.55), Species)
```

1. Rewrite using baseR chaining operator

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr       1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
iris |>
  as_tibble() |>
  select(starts_with("Petal"), Species) |>
  filter(Petal.Length<1.55) |>
  arrange(Species)
```

```
# A tibble: 37 x 3
```

```
  Petal.Length Petal.Width Species
    <dbl>         <dbl> <fct>
```

```

1          1.4          0.2 setosa
2          1.4          0.2 setosa
3          1.3          0.2 setosa
4          1.5          0.2 setosa
5          1.4          0.2 setosa
6          1.4          0.3 setosa
7          1.5          0.2 setosa
8          1.4          0.2 setosa
9          1.5          0.1 setosa
10         1.5          0.2 setosa
# i 27 more rows

```

2. What is meant by long format data and wide format data?

1. Long format is a data table organization where each row is one observation and each column is a variable. Repeated measures are stored in additional rows. This is the ideal format when working with structured data for statistical analysis.
2. Wide format is where there can be multiple observations per row. Repeated measures are stored in additional columns instead of additional rows. Sometimes this makes the data easier to read in a table format.
3. Which do we generally prefer for statistical analysis? We generally prefer the long format to do analyses on. This enables us to do analysis on the variables and include each observation separately, not necessarily each individual though as some rows may be repeated measures on the same individual.

Task 2

Glass Data

Read in the glass data to a tibble. Change the variable Type to be a factor variable with character names and filter the rows to include only the tableware and headlamps type and Fe less than 0.2.

```

#Read in the glass data to a tibble
glass_data <- read_csv("https://www4.stat.ncsu.edu/online/datasets/glass.data", col_names=c(

```

```

Rows: 214 Columns: 11

```

```

-- Column specification -----

```

```

Delimiter: ","

```

```

dbl (11): Id, RI, Na, Mg, Al, Si, K, Ca, Ba, Fe, Type

```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
glass_data
```

```
# A tibble: 214 x 11
      Id    RI    Na    Mg    Al    Si    K    Ca    Ba    Fe  Type
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     1  1.52  13.6  4.49  1.1  71.8  0.06  8.75    0  0      1
2     2  1.52  13.9  3.6   1.36  72.7  0.48  7.83    0  0      1
3     3  1.52  13.5  3.55  1.54  73.0  0.39  7.78    0  0      1
4     4  1.52  13.2  3.69  1.29  72.6  0.57  8.22    0  0      1
5     5  1.52  13.3  3.62  1.24  73.1  0.55  8.07    0  0      1
6     6  1.52  12.8  3.61  1.62  73.0  0.64  8.07    0  0.26    1
7     7  1.52  13.3  3.6   1.14  73.1  0.58  8.17    0  0      1
8     8  1.52  13.2  3.61  1.05  73.2  0.57  8.24    0  0      1
9     9  1.52  14.0  3.58  1.37  72.1  0.56  8.3     0  0      1
10    10  1.52  13    3.6   1.36  73.0  0.57  8.4     0  0.11    1
# i 204 more rows
```

```
#Change the variable Type to be a factor and filter the rows
```

```
glass_data <- glass_data |>
  mutate(Type = factor(Type, levels=c(1,2,3,4,5,6,7), labels= c('building_windows_float_proce
  filter(Fe<0.2, Type %in% c('tableware' , 'headlamps'))
glass_data
```

```
# A tibble: 38 x 11
      Id    RI    Na    Mg    Al    Si    K    Ca    Ba    Fe  Type
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct>
1   177  1.52  14    2.39  1.56  72.4  0    9.57  0    0 tableware
2   178  1.52  13.8  2.41  1.19  72.8  0    9.77  0    0 tableware
3   179  1.52  14.5  2.24  1.62  72.4  0    9.26  0    0 tableware
4   180  1.52  14.1  2.19  1.66  72.7  0    9.32  0    0 tableware
5   181  1.51  14.4  1.74  1.54  74.6  0    7.59  0    0 tableware
6   182  1.52  15.0  0.78  1.74  72.5  0    9.95  0    0 tableware
7   183  1.52  14.2  0     2.09  72.7  0   10.9  0    0 tableware
8   184  1.52  14.6  0     0.56  73.5  0   11.2  0    0 tableware
9   185  1.51  17.4  0     0.34  75.4  0    6.65  0    0 tableware
10  186  1.51  13.7  3.2   1.81  72.8  1.76  5.43  1.19  0 headlamps
# i 28 more rows
```

Yeast Data

Read Yeast data into a tibble and add column names. Remove the columns 'Sequence' and 'nuc' by chaining. Add a mean and median column for each numerical variable. Group each mean and median for each variable by the class variable.

```
# Read data into tibble
```

```
yeast_data <- read_fwf('https://www4.stat.ncsu.edu/online/datasets/yeast.data', fwf_widths(c
```

```
Rows: 1484 Columns: 10
```

```
-- Column specification -----
```

```
chr (2): Sequence, class
```

```
dbl (8): mcg, gvh, alm, mit, erl, pox, vac, nuc
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
yeast_data
```

```
# A tibble: 1,484 x 10
```

	Sequence <chr>	mcg <dbl>	gvh <dbl>	alm <dbl>	mit <dbl>	erl <dbl>	pox <dbl>	vac <dbl>	nuc <dbl>	class <chr>
1	ADT1_YEAST	0.58	0.61	0.47	0.13	0.5	0	0.48	0.22	MIT
2	ADT2_YEAST	0.43	0.67	0.48	0.27	0.5	0	0.53	0.22	MIT
3	ADT3_YEAST	0.64	0.62	0.49	0.15	0.5	0	0.53	0.22	MIT
4	AAR2_YEAST	0.58	0.44	0.57	0.13	0.5	0	0.54	0.22	NUC
5	AATM_YEAST	0.42	0.44	0.48	0.54	0.5	0	0.48	0.22	MIT
6	AATC_YEAST	0.51	0.4	0.56	0.17	0.5	0.5	0.49	0.22	CYT
7	ABC1_YEAST	0.5	0.54	0.48	0.65	0.5	0	0.53	0.22	MIT
8	BAF1_YEAST	0.48	0.45	0.59	0.2	0.5	0	0.58	0.34	NUC
9	ABF2_YEAST	0.55	0.5	0.66	0.36	0.5	0	0.49	0.22	MIT
10	ABP1_YEAST	0.4	0.39	0.6	0.15	0.5	0	0.58	0.3	CYT

```
# i 1,474 more rows
```

```
#Remove sequence and nuc columns and add mean and median for each column grouped by class
```

```
yeast_data <- yeast_data |>  
  select(mcg:vac, class) |>  
  group_by(class) |>
```

```
mutate(across(where(is.numeric), c(mean=mean, median=median), .names='{.fn}_{.col}'))

yeast_data
```

```
# A tibble: 1,484 x 22
# Groups:   class [10]
   mcg   gvh   alm   mit   erl   pox   vac class mean_mcg median_mcg mean_gvh
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>    <dbl>      <dbl>    <dbl>
1  0.58  0.61  0.47  0.13  0.5   0    0.48 MIT      0.521      0.51    0.533
2  0.43  0.67  0.48  0.27  0.5   0    0.53 MIT      0.521      0.51    0.533
3  0.64  0.62  0.49  0.15  0.5   0    0.53 MIT      0.521      0.51    0.533
4  0.58  0.44  0.57  0.13  0.5   0    0.54 NUC      0.452      0.45    0.456
5  0.42  0.44  0.48  0.54  0.5   0    0.48 MIT      0.521      0.51    0.533
6  0.51  0.4   0.56  0.17  0.5   0.5  0.49 CYT      0.481      0.48    0.470
7  0.5   0.54  0.48  0.65  0.5   0    0.53 MIT      0.521      0.51    0.533
8  0.48  0.45  0.59  0.2   0.5   0    0.58 NUC      0.452      0.45    0.456
9  0.55  0.5   0.66  0.36  0.5   0    0.49 MIT      0.521      0.51    0.533
10 0.4   0.39  0.6   0.15  0.5   0    0.58 CYT      0.481      0.48    0.470
# i 1,474 more rows
# i 11 more variables: median_gvh <dbl>, mean_alm <dbl>, median_alm <dbl>,
#   mean_mit <dbl>, median_mit <dbl>, mean_erl <dbl>, median_erl <dbl>,
#   mean_pox <dbl>, median_pox <dbl>, mean_vac <dbl>, median_vac <dbl>
```

Wine Data

Load the wine data and combine the red and white wine datasets using the streamlined version.

```
library(readxl)
library(here)
```

Warning: package 'here' was built under R version 4.5.3

here() starts at C:/Users/cbhut/OneDrive/Documents/GitHub/Data-Science-ED-Group

```
wine_names <- read_excel(here("HW_3", "white-wine.xlsx"), sheet = 2)

white_wine_data <- read_excel(here("HW_3", "white-wine.xlsx")) |>
```

```

setNames(wine_names$Variables) |>
mutate(type = "White")

red_wine <- read_delim("https://www4.stat.ncsu.edu/~online/datasets/red-wine.csv",delim = ";",
  setNames(wine_names$Variables) |>
  mutate(type = "Red")

```

Rows: 1599 Columns: 12

```

-- Column specification -----
Delimiter: ";"
dbl (12): fixed_acidity, volatile_acidity, citric_acid, residual_sugar, chlo...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

```

wine_data <- bind_rows(white_wine_data, red_wine)
wine_data

```

A tibble: 6,497 x 13

	fixed_acidity	volatile_acidity	citric_acid	residual_sugar	chlorides
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	7	0.27	0.36	20.7	0.045
2	6.3	0.3	0.34	1.6	0.049
3	8.1	0.28	0.4	6.9	0.05
4	7.2	0.23	0.32	8.5	0.058
5	7.2	0.23	0.32	8.5	0.058
6	8.1	0.28	0.4	6.9	0.05
7	6.2	0.32	0.16	7	0.045
8	7	0.27	0.36	20.7	0.045
9	6.3	0.3	0.34	1.6	0.049
10	8.1	0.22	0.43	1.5	0.044

i 6,487 more rows

```

# i 8 more variables: free_sulfur_dioxide <dbl>, total_sulfur_dioxide <dbl>,
#   density <dbl>, pH <dbl>, sulphates <dbl>, alcohol <dbl>, quality <dbl>,
#   type <chr>

```

Filter the data to look only at higher quality wines and reasonable alcohol content, then select the variables alcohol, type, quality and any variable that contains acid. Then group the data by quality and calculate the mean and standard deviation for alcohol for each level of quality.

```
wine_data <- wine_data |>
  filter(quality>6.5, alcohol<13.2) |>
  arrange(desc(quality)) |>
  select(contains('acid'), alcohol, type, quality) |>
  group_by(quality) |>
  mutate(across(alcohol, c(mean=mean, std= sd), .names='{.fn}_{.col}'))
```

Database Practice

Connect to the database and open all the tables in the database.

```
#install.packages("RSQLite")
library(RSQLite)
```

Warning: package 'RSQLite' was built under R version 4.5.3

```
library(DBI)
getwd()
```

```
[1] "C:/Users/cbhut/OneDrive/Documents/GitHub/Data-Science-ED-Group/HW_4"
```

```
setwd("~/GitHub/Data-Science-ED-Group/HW_4")
con <- dbConnect(RSQLite::SQLite(), "lahman.db")
dbListTables(con)
```

```
[1] "AllstarFull"      "Appearances"      "AwardsManagers"
[4] "AwardsPlayers"    "AwardsShareManagers" "AwardsSharePlayers"
[7] "Batting"          "BattingPost"      "CollegePlaying"
[10] "Fielding"         "FieldingOF"       "FieldingOFsplit"
[13] "FieldingPost"     "HallOfFame"       "HomeGames"
[16] "LahmanData"       "Managers"         "ManagersHalf"
[19] "Parks"           "People"           "Pitching"
[22] "PitchingPost"     "Salaries"         "Schools"
[25] "SeriesPost"       "Teams"            "TeamsFranchises"
[28] "TeamsHalf"       "battingLabels"    "fieldingLabels"
[31] "pitchingLabels"
```

Use the `tbl()` function and `dplyr` to return all of the data from the Teams table for the year 2015

```
teams <- tbl(con, "Teams") |>
  filter(yearID == 2015) |>
  show_query()
```

```
<SQL>
SELECT `Teams`.*
FROM `Teams`
WHERE (`yearID` = 2015.0)
```

Use SQL to filter the data

```
tbl(con, sql(
  "SELECT `Teams`.*
  FROM `Teams`
  WHERE (`yearID` = 2015.0)"
))
```

```
# Source:   SQL [?? x 48]
# Database: sqlite 3.53.3 [C:\Users\cbhut\OneDrive\Documents\GitHub\Data-
Science-ED-Group\HW_4\lahman.db]
  yearID lgID  teamID franchID divID  Rank    G Ghome    W    L DivWin WCWin
   <int> <chr>  <chr>   <chr>    <chr> <int>  <int> <int> <int> <int> <chr>  <chr>
1   2015  NL    ARI     ARI      W      3    162    81    79    83 N      N
2   2015  NL    ATL     ATL      E      4    162    81    67    95 N      N
3   2015  AL    BAL     BAL      E      3    162    78    81    81 N      N
4   2015  AL    BOS     BOS      E      5    162    81    78    84 N      N
5   2015  AL    CHA     CHW      C      4    162    81    76    86 N      N
6   2015  NL    CHN     CHC      C      3    162    81    97    65 N      Y
7   2015  NL    CIN     CIN      C      5    162    81    64    98 N      N
8   2015  AL    CLE     CLE      C      3    161    80    81    80 N      N
9   2015  NL    COL     COL      W      5    162    81    68    94 N      N
10  2015  AL    DET     DET      C      5    161    81    74    87 N      N
# i more rows
# i 36 more variables: LgWin <chr>, WSWin <chr>, R <int>, AB <int>, H <int>,
# X2B <int>, X3B <int>, HR <int>, BB <int>, SO <int>, SB <int>, CS <int>,
# HBP <int>, SF <int>, RA <int>, ER <int>, ERA <dbl>, CG <int>, SHO <int>,
# SV <int>, IPouts <int>, HA <int>, HRA <int>, BBA <int>, SOA <int>, E <int>,
# DP <int>, FP <dbl>, name <chr>, park <chr>, attendance <int>, BPF <int>,
# PPF <int>, teamIDBR <chr>, teamIDlahman45 <chr>, teamIDretro <chr>
```

Return all of the players in the hall of fame, the year they were voted into the hall of fame, and their category (only those three variables!). See the HallOfFame table, the inducted variable is important here.

```
fame <- tbl(con, "HallofFame") |>
  filter(inducted == 'Y') |>
  select(playerID, yearID, category)

fame
```

```
# Source:   SQL [?? x 3]
# Database: sqlite 3.53.3 [C:\Users\cbhut\OneDrive\Documents\GitHub\Data-
Science-ED-Group\HW_4\lahman.db]
   playerID  yearID category
   <chr>      <int> <chr>
1 cobbty01   1936 Player
2 ruthba01   1936 Player
3 wagneho01  1936 Player
4 mathech01  1936 Player
5 johnswa01  1936 Player
6 lajoina01  1937 Player
7 speaktr01 1937 Player
8 youngcy01  1937 Player
9 bulkemo99  1937 Pioneer/Executive
10 johnsba99 1937 Pioneer/Executive
# i more rows
```

Combine the fame with the People table in order to have the nameFirst and nameLast names added to what was returned above.

```
#tbl(con, "People")
name_fame <- inner_join(tbl(con, "HallofFame") |> filter(inducted=="Y") |>
  select(playerID, yearID, category),
  tbl(con, "People") |>
  select(playerID, nameFirst, nameLast),
  by=join_by(playerID)) |>
  collect()
name_fame
```

```
# A tibble: 338 x 5
   playerID  yearID category nameFirst nameLast
```

	<chr>	<int>	<chr>	<chr>	<chr>
1	cobbty01	1936	Player	Ty	Cobb
2	ruthba01	1936	Player	Babe	Ruth
3	wagneho01	1936	Player	Honus	Wagner
4	mathech01	1936	Player	Christy	Mathewson
5	johnswa01	1936	Player	Walter	Johnson
6	lajoina01	1937	Player	Nap	Lajoie
7	speaktr01	1937	Player	Tris	Speaker
8	youngcy01	1937	Player	Cy	Young
9	bulkemo99	1937	Pioneer/Executive	Morgan	Bulkeley
10	johnsba99	1937	Pioneer/Executive	Ban	Johnson

i 328 more rows

Return only the playerID (manager ID in this case), G, W, and L columns from the Managers table. Use chaining to then:

- Determine the overall win/loss records (sum of wins and sum of losses) for each of these hall of fame managers.

```
managers <- tbl(con, "Managers") |>
  select(playerID, G, W, L) |>
  group_by(playerID) |>
  summarize(G_managed = sum(G, na.rm = TRUE),
Total_W = sum(W, na.rm = TRUE),
Total_L = sum(L, na.rm = TRUE)) |>
  collect()
managers
```

```
# A tibble: 749 x 4
  playerID  G_managed Total_W Total_L
  <chr>      <int>    <int>  <int>
1 actama99     890     372    518
2 adairbi99     10      4      6
3 adcocjo01    162     75     87
4 addybo01     31      8     23
5 allenbo01    179     87     87
6 allisdo01     23      2     21
7 alomasa02     13      8      5
8 aloufe01    2053    1033   1020
9 alstowa01    3658    2040   1613
10 altobjo01    844     437    407
# i 739 more rows
```

Create a new variable that is the career win/loss percentage (Total_W/G_managed). (Note: I did this after collect() otherwise the column type has to be set. . .) • Sort the resulting data by the win/loss percentage variable (from largest to smallest)

```
managers <- managers |>
  mutate(career_win_loss_percentage=Total_W/G_managed) |>
  arrange(desc(career_win_loss_percentage))
managers
```

```
# A tibble: 749 x 5
  playerID  G_managed Total_W Total_L career_win_loss_percentage
  <chr>      <int>    <int>  <int>                <dbl>
1 bensove01      1      1      0                1
2 burwebi01      1      1      0                1
3 cohenan01      1      1      0                1
4 ebeldi99       3      3      0                1
5 falkbi01       1      1      0                1
6 hardeme01      3      3      0                1
7 simmote01      1      1      0                1
8 steinte01      2      2      0                1
9 sukefc101      2      2      0                1
10 tamarjo01     1      1      0                1
# i 739 more rows
```

Use the results from the previous two questions to answer this one! Return information for only the people that managed a team and were inducted into the hall of fame (regardless of their category in the hall of fame - you want to return all people in the hall of fame that ever managed at all). (Just return all of the variables from both of the above two questions - 9 total columns)

```
inner_join(managers,
  name_fame,
  by = join_by(playerID)) |>
collect()
```

```
# A tibble: 97 x 9
  playerID  G_managed Total_W Total_L career_win_loss_percent~1 yearID category
  <chr>      <int>    <int>  <int>                <dbl>  <int> <chr>
1 simmote01      1      1      0                1      2020 Player
2 wrighge01     85     59     25             0.694   1937 Pioneer~
3 spaldal01    126     78     47             0.619   1939 Pioneer~
```

4	mccarjo99	3487	2125	1333	0.609	1957	Manager
5	comisch01	1410	840	541	0.596	1939	Pioneer~
6	southbi01	1770	1044	704	0.590	2008	Manager
7	seleefr99	2180	1284	862	0.589	1999	Manager
8	chancfr01	1622	946	648	0.583	1946	Player
9	weaveea99	2541	1480	1060	0.582	1996	Manager
10	lopezal01	2425	1410	1004	0.581	1977	Manager

i 87 more rows

i abbreviated name: 1: career_win_loss_percentage

i 2 more variables: nameFirst <chr>, nameLast <chr>

,